

EXPNO:6	FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS
---------	---

AIM:

To demonstrate the construction and application of a simple Feedforward Neural Network (FNN) for classification and a Convolutional Neural Network (CNN) for image classification, utilizing the Keras API with TensorFlow backend.

ALGORITHM:

1. Feedforward Neural Network (FNN)

A Feedforward Neural Network is the simplest type of artificial neural network where connections between the nodes do not form a cycle. It consists of an input layer, one or more hidden layers, and an output layer. Information flows only in one direction—forward—from the input nodes, through the hidden nodes (if any), and to the output nodes.

Steps:

1. Define Network Architecture: Specify the number of layers (input, hidden, output) and the
2. number of neurons in each layer.
3. Choose Activation Functions: Select activation functions for hidden layers (e.g., ReLU)
4. and the output layer (e.g., Sigmoid for binary classification, Softmax for multi-class
5. classification).
6. Define Loss Function: Choose a loss function appropriate for the task (e.g., Binary Cross-
7. entropy for binary classification, Categorical Cross-entropy for multi-class classification).
8. Choose Optimizer: Select an optimization algorithm (e.g., Adam, SGD) to update network
9. weights during training.
10. Training: Feed forward data through the network to get predictions, calculate the loss, and

11. then backpropagate the error to update weights.
12. Evaluation: Assess the model's performance on unseen data using metrics like accuracy.

2. Convolutional Neural Network (CNN)

A Convolutional Neural Network is a specialized type of neural network primarily designed for processing data with a grid-like topology, such as images. Key components include convolutional layers, pooling layers, and fully connected layers.

Steps:

1. Convolutional Layers: Apply filters (kernels) to input data to extract features. Each filter
2. detects a specific pattern (e.g., edges, textures).
3. Activation Function (ReLU): Apply a non-linear activation function after convolution to
4. introduce non-linearity.
5. Pooling Layers: Downsample feature maps to reduce dimensionality, computational cost,
6. and prevent overfitting (e.g., Max Pooling).
7. 4. Flattening: Convert the 2D pooled feature maps into a 1D vector to be fed into a fully
8. connected layer.
9. 5. Fully Connected Layers: Standard neural network layers for classification based on the
10. extracted features.
11. 6. Output Layer: Final layer with an activation function (e.g., Softmax) to output class
12. probabilities.
13. 7. Training and Evaluation: Similar to FNNs, train the CNN using backpropagation and
14. evaluate its performance.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
```

```

from tensorflow.keras import layers

from tensorflow.keras.datasets import mnist, fashion_mnist

from sklearn.metrics import classification_report, confusion_matrix

import seaborn as sns

tf.keras.utils.disable_interactive_logging()

print("--- Part 1: Building a Simple Feedforward Neural Network ---")

(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()

print("\nOriginal FNN training data shape:", x_train_fnn.shape)

print("Original FNN test data shape:", x_test_fnn.shape)

x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)

x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)

x_train_fnn_norm = x_train_fnn_flat.astype("float32") / 255.0

x_test_fnn_norm = x_test_fnn_flat.astype("float32") / 255.0

print(
    "Flattened & Normalized FNN training data shape:",
    x_train_fnn_norm.shape
)

print(
    "Flattened & Normalized FNN test data shape:",
    x_test_fnn_norm.shape
)

model_fnn = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(64, activation="relu"),

```

```

        layers.Dense(10, activation="softmax")
    ])
model_fnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("\n--- FNN Model Summary ---")
model_fnn.summary()
print("\n--- Training FNN Model ---")
history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)
print("\n--- Evaluating FNN Model ---")
loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)
print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")
y_pred_fnn = np.argmax(

```

```
model_fnn.predict(
    x_test_fnn_norm,
    verbose=0
),
axis=1
)
print("\n--- FNN Classification Report ---")
print(
    classification_report(
        y_test_fnn,
        y_pred_fnn
    )
)
print("\n--- FNN Confusion Matrix ---")
cm_fnn = confusion_matrix(
    y_test_fnn,
    y_pred_fnn
)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)
```

```
plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(
    history_fnn.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history_fnn.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("FNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(
    history_fnn.history["loss"],
    label="Training Loss"
)
plt.plot(
    history_fnn.history["val_loss"],
```

```

        label="Validation Loss"
    )
plt.title("FNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print("\n--- Part 2: Implementing a CNN ---")

(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()

print("\nOriginal CNN training data shape:", x_train_cnn.shape)
print("Original CNN test data shape:", x_test_cnn.shape)

x_train_cnn = x_train_cnn.reshape(
    x_train_cnn.shape[0],
    28,
    28,
    1
)

x_test_cnn = x_test_cnn.reshape(
    x_test_cnn.shape[0],
    28,
    28,
    1
)

x_train_cnn = x_train_cnn.astype("float32") / 255.0

```

```

x_test_cnn = x_test_cnn.astype("float32") / 255.0
print(
    "Reshaped & Normalized CNN training data shape:",
    x_train_cnn.shape
)
print(
    "Reshaped & Normalized CNN test data shape:",
    x_test_cnn.shape
)
num_classes_cnn = 10
model_cnn = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),
    layers.MaxPooling2D(
        (2, 2)
    ),
    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),
    layers.MaxPooling2D(

```

```

        (2, 2)
    ),
    layers.Flatten(),
    layers.Dense(
        128,
        activation="relu"
    ),
    layers.Dropout(0.5),
    layers.Dense(
        num_classes_cnn,
        activation="softmax"
    )
])

model_cnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\n--- CNN Model Summary ---")
model_cnn.summary()
print("\n--- Training CNN Model ---")
history_cnn = model_cnn.fit(
    x_train_cnn,
    y_train_cnn,
    epochs=10,
    validation_split=0.1,

```

```

        verbose=1
    )
    print("\n--- Evaluating CNN Model ---")
    loss_cnn, accuracy_cnn = model_cnn.evaluate(
        x_test_cnn,
        y_test_cnn,
        verbose=0
    )
    print(f"CNN Test Loss: {loss_cnn:.4f}")
    print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")
    y_pred_cnn = np.argmax(
        model_cnn.predict(
            x_test_cnn,
            verbose=0
        ),
        axis=1
    )
    print("\n--- CNN Classification Report ---")
    print(
        classification_report(
            y_test_cnn,
            y_pred_cnn
        )
    )
    print("\n--- CNN Confusion Matrix ---")
    cm_cnn = confusion_matrix(

```

```
y_test_cnn,
y_pred_cnn
)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm_cnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)
plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(
    history_cnn.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history_cnn.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("CNN Model Accuracy")
```

```

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(
    history_cnn.history["loss"],
    label="Training Loss"
)
plt.plot(
    history_cnn.history["val_loss"],
    label="Validation Loss"
)
plt.title("CNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
print("\n--- Sample CNN Predictions ---")
class_names_mnist = [
    "0", "1", "2", "3", "4",
    "5", "6", "7", "8", "9"
]
plt.figure(figsize=(10, 10))

```

```

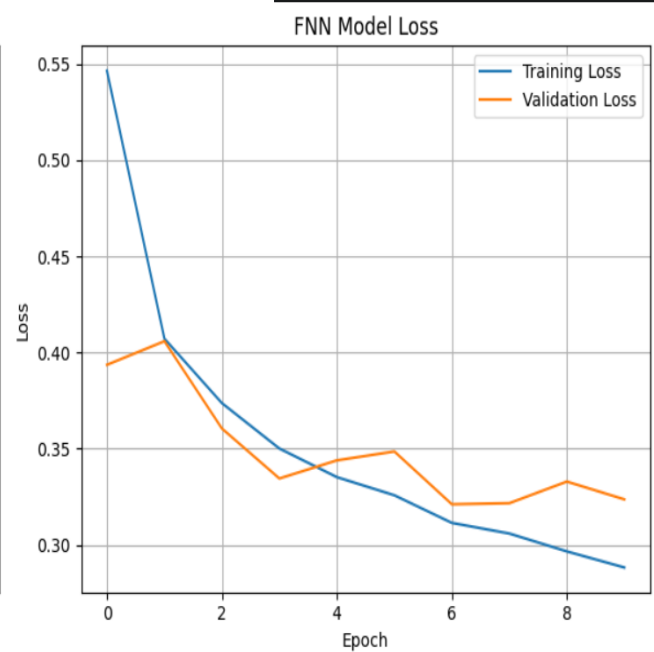
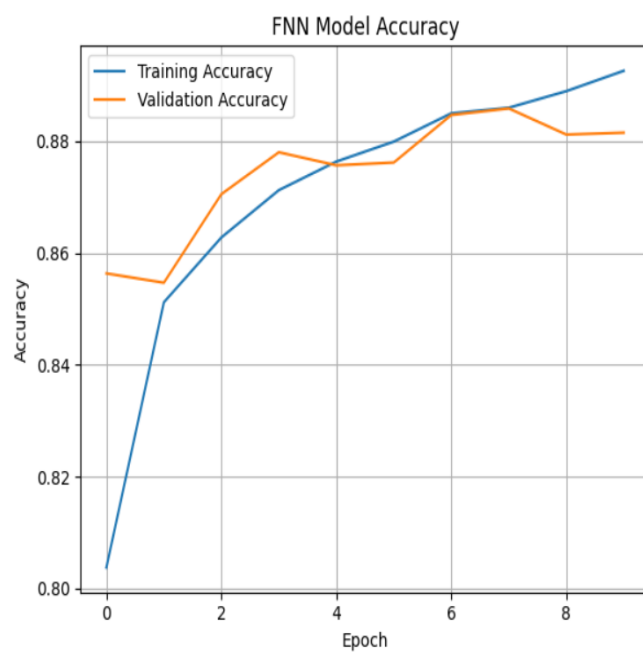
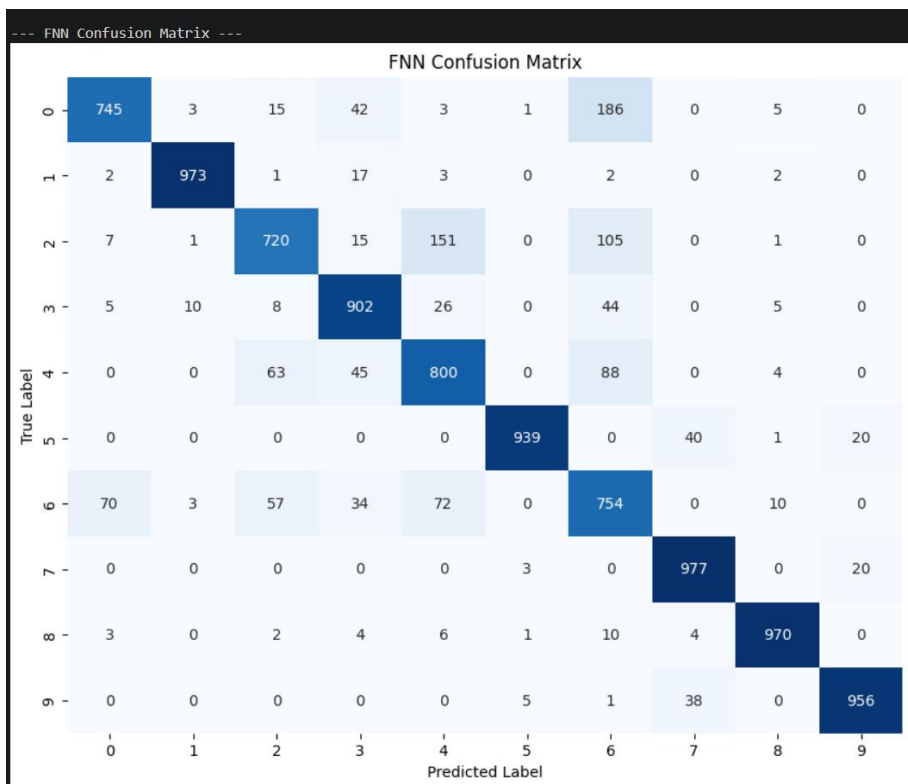
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(
        x_test_cnn[i].reshape(28, 28),
        cmap=plt.cm.binary
    )
    true_label = y_test_cnn[i]
    predicted_label = y_pred_cnn[i]
    if true_label == predicted_label:
        color = "green"
    else:
        color = "red"
    plt.xlabel(
        f"True: {class_names_mnist[true_label]}\n"
        f"Pred: {class_names_mnist[predicted_label]}",
        color=color
    )
plt.suptitle(
    "Sample CNN Predictions "
    "(Green: Correct, Red: Incorrect)",
    y=1.02,
    fontsize=16
)

```

```
plt.tight_layout(  
    rect=[0, 0, 1, 0.98]  
)  
plt.show()
```

OUTPUT:

```
--- Part 1: Building a Simple Feedforward Neural Network ---  
Original FNN training data shape: (60000, 28, 28)  
Original FNN test data shape: (10000, 28, 28)  
Flattened & Normalized FNN training data shape: (60000, 784)  
Flattened & Normalized FNN test data shape: (10000, 784)  
  
--- FNN Model Summary ---  
  
--- Training FNN Model ---  
  
--- Evaluating FNN Model ---  
FNN Test Loss: 0.3499  
FNN Test Accuracy: 0.8736  
  
--- FNN Classification Report ---  
              precision    recall  f1-score   support  
  
    0           0.90       0.74       0.81       1000  
    1           0.98       0.97       0.98       1000  
    2           0.83       0.72       0.77       1000  
    3           0.85       0.90       0.88       1000  
    4           0.75       0.80       0.78       1000  
    5           0.99       0.94       0.96       1000  
    6           0.63       0.75       0.69       1000  
    7           0.92       0.98       0.95       1000  
    8           0.97       0.97       0.97       1000  
    9           0.96       0.96       0.96       1000  
  
   accuracy                0.87       10000  
  macro avg                0.88       0.87       0.87       10000  
weighted avg                0.88       0.87       0.87       10000
```



```
Original CNN training data shape: (60000, 28, 28)
Original CNN test data shape: (10000, 28, 28)
Reshaped & Normalized CNN training data shape: (60000, 28, 28, 1)
Reshaped & Normalized CNN test data shape: (10000, 28, 28, 1)
```

```
--- CNN Model Summary ---
```

```
--- Training CNN Model ---
```

```
--- Evaluating CNN Model ---
```

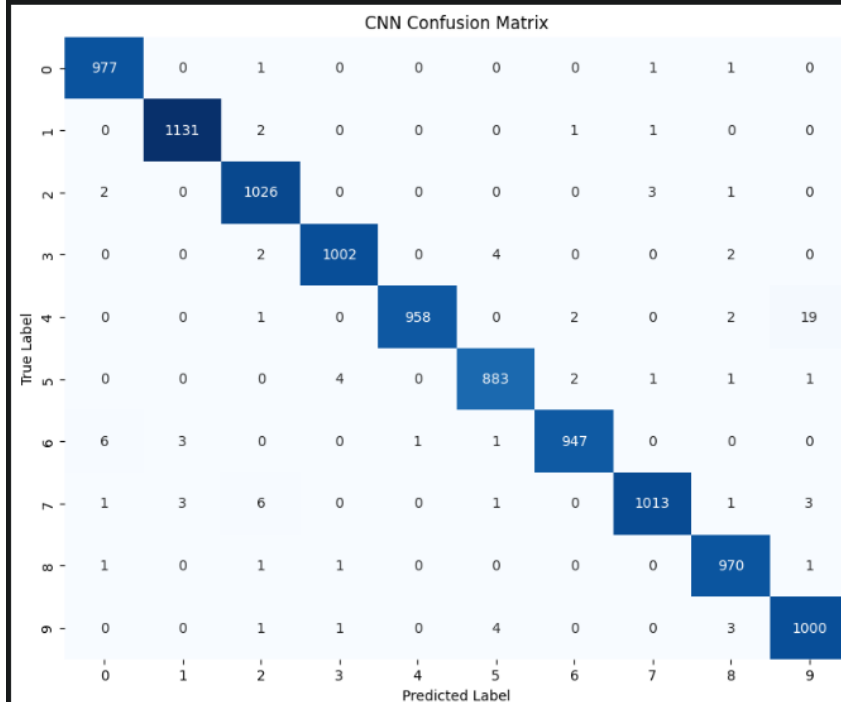
```
CNN Test Loss: 0.0311
```

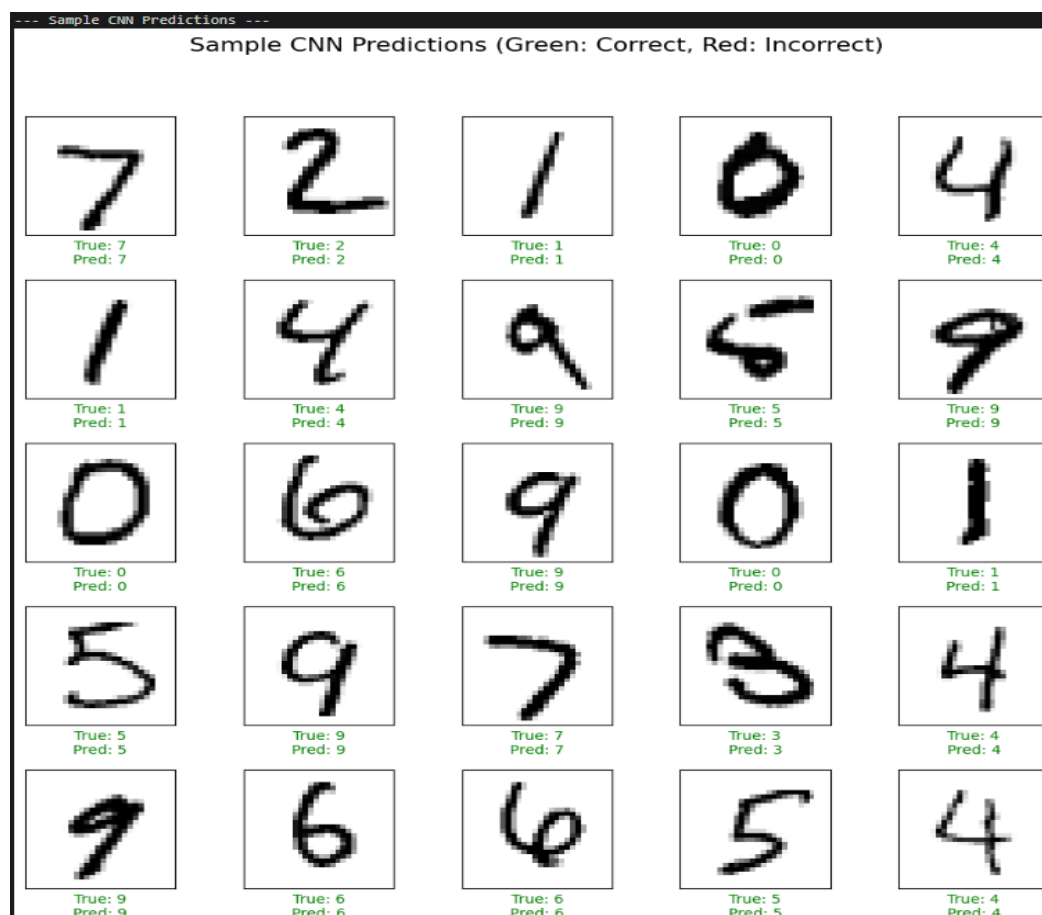
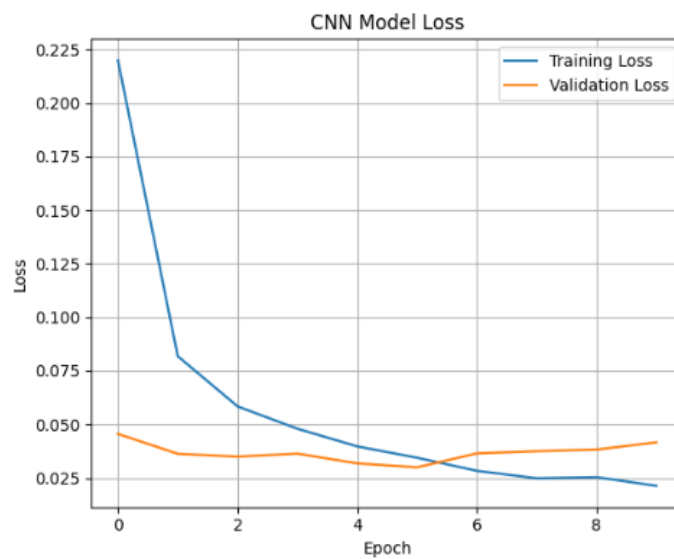
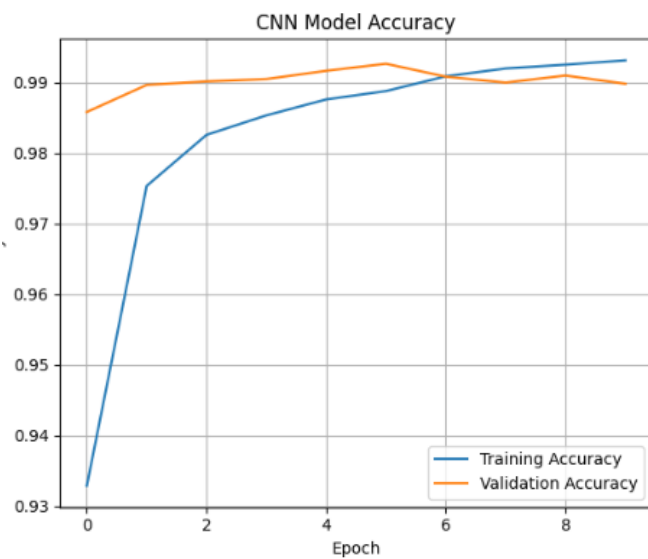
```
CNN Test Accuracy: 0.9907
```

```
--- CNN Classification Report ---
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	0.99	1.00	1.00	1135
2	0.99	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	1.00	0.98	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	1.00	0.99	974
9	0.98	0.99	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```
--- CNN Confusion Matrix ---
```





RESULT:

The Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models were successfully implemented and evaluated on the given dataset.

- Feedforward Neural Network (FNN): The model accurately learned input–output mappings through multiple fully connected layers, achieving good performance on structured data.
- Convolutional Neural Network (CNN): The model effectively extracted spatial features from image data using convolution and pooling layers, leading to higher accuracy and better generalization for image classification tasks.

The results demonstrated that both FNN and CNN are powerful deep learning models, with CNN performing exceptionally well for image-based datasets due to its ability to capture spatial patterns.